

```

import React, { useState, useRef, useEffect, useCallback } from "react";
import { HelpCircle, LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// ⓘ LOCAL TYPE ISOLATION GATEWAY
export interface LabelProps extends React.LabelHTMLAttributes<HTMLLabelElement> {
  isRequired?: boolean;
  error?: string;
  disabled?: boolean;
  hintContent?: React.ReactNode;
  hintIcon?: LucideIcon;
}

export const Label = React.forwardRef<HTMLLabelElement, LabelProps>(
  (
    {
      isRequired = false,
      error,
      disabled = false,
      hintContent,
      hintIcon: HintIcon = HelpCircle,
      className,
      children,
      ...props
    },
    ref,
  ) => {
    const hasError = Boolean(error);
    const isInteractive = !disabled && !hasError;
    const [isHintOpen, setIsHintOpen] = useState(false);
    const [currentSide, setCurrentSide] = useState<"top" | "bottom">("top");
    const triggerRef = useRef<HTMLButtonElement>(null);
    const contentRef = useRef<HTMLDivElement>(null);
    const enterTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
    const leaveTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
    const positionStylesRef = useRef<React.CSSProperties>({});

    const DEFAULT_ENTER_DELAY = 200;
    const DEFAULT_LEAVE_DELAY = 150;

    const updatePosition = useCallback(() => {
      if (triggerRef.current && contentRef.current) {
        const trigger = triggerRef.current.getBoundingClientRect();
        const content = contentRef.current.getBoundingClientRect();

```

```

const viewportWidth = window.innerWidth;
const viewportHeight = window.innerHeight;
const gap = 8;
const viewportPadding = 8;

let top = trigger.bottom + gap;
let left = trigger.left + (trigger.width - content.width) / 2;
let side: "top" | "bottom" = "bottom";

if (top + content.height > viewportHeight - viewportPadding) {
  top = trigger.top - content.height - gap;
  side = "top";
}
if (left < viewportPadding) {
  left = viewportPadding;
}
if (left + content.width > viewportWidth - viewportPadding) {
  left = viewportWidth - content.width - viewportPadding;
}

setCurrentSide(side);

positionStylesRef.current = {
  top: `${top}px`,
  left: `${left}px`,
  transform: "translateX(-50%)",
};
}, []);

useEffect(() => {
  if (isHintOpen) {
    updatePosition();
    window.addEventListener("resize", updatePosition);
    window.addEventListener("scroll", updatePosition, true);
  }
  return () => {
    window.removeEventListener("resize", updatePosition);
    window.removeEventListener("scroll", updatePosition, true);
  };
}, [isHintOpen, updatePosition]);

const clearTimeouts = useCallback(() => {
  if (enterTimeoutRef.current) {
    clearTimeout(enterTimeoutRef.current);
  }
  if (leaveTimeoutRef.current) {
    clearTimeout(leaveTimeoutRef.current);
  }
}, []);

```

```

const handleTriggerEnter = useCallback(() => {
  if (!isInteractive) return;
  clearTimeouts();
  enterTimeoutRef.current = setTimeout(() => {
    setIsHintOpen(true);
  }, DEFAULT_ENTER_DELAY);
}, [clearTimeouts, isInteractive]);

const handleTriggerLeave = useCallback(() => {
  clearTimeouts();
  leaveTimeoutRef.current = setTimeout(() => {
    setIsHintOpen(false);
  }, DEFAULT_LEAVE_DELAY);
}, [clearTimeouts]);

const handleContentEnter = useCallback(() => {
  clearTimeouts();
  setIsHintOpen(true);
}, [clearTimeouts]);

const handleContentLeave = useCallback(() => {
  clearTimeouts();
  leaveTimeoutRef.current = setTimeout(() => {
    setIsHintOpen(false);
  }, DEFAULT_LEAVE_DELAY);
}, [clearTimeouts]);

const handleKeyDown = useCallback((e: React.KeyboardEvent) => {
  if (e.key === "Escape") {
    setIsHintOpen(false);
  }
}, []);

return (
  <label
    ref={ref}
    className={cn(
      "inline-flex items-center gap-1.5",
      "font-sans text-sm font-medium leading-none",
      "transition-colors duration-200 ease-out",
      "text-text-main",
      hasError && "text-destructive/90",
      disabled && "opacity-50 pointer-events-none cursor-not-allowed",
      className,
    )}
    {...props}
  >
    {children}
    {isRequired && (

```

```

<span
  className={cn(
    "inline-flex items-center justify-center",
    "text-destructive",
    "transition-opacity duration-200 ease-out",
    "aria-hidden",
  )}
  aria-hidden="true"
>
  *
</span>
)}
{hintContent && (
  <span className="relative inline-flex items-center justify-center">
    <button
      ref={triggerRef}
      type="button"
      className={cn(
        "relative flex items-center justify-center",
        "w-5 h-5 rounded-md",
        "text-muted-foreground/60 hover:text-foreground",
        "bg-transparent hover:bg-accent",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
        "disabled:opacity-50 disabled:pointer-events-none",
        !isInteractive &&
        "opacity-50 pointer-events-none cursor-not-allowed",
      )}
      aria-label="Help"
      aria-describedby={isHintOpen ? "hint-tooltip" : undefined}
      aria-expanded={isHintOpen}
      disabled={!isInteractive}
      onMouseEnter={handleTriggerEnter}
      onMouseLeave={handleTriggerLeave}
      onFocus={handleTriggerEnter}
      onBlur={handleTriggerLeave}
      onKeyDown={handleKeyDown}
    >
      <HintIcon className="w-3.5 h-3.5" aria-hidden="true" />
    </button>
    {isHintOpen && (
      <div
        ref={contentRef}
        id="hint-tooltip"
        role="tooltip"
        className={cn(
          "fixed z-50 pointer-events-auto",
          "bg-card backdrop-blur-[var(--backdrop-blur)]",

```



```
Label.displayName = "Label";
```